

LeetCode 1504: Count Submatrices With All Ones

Author: Arpit Raj

Date: July 9, 2026

Institution: LNMIIT Jaipur

Topic: Monotonic Stack / DP / Histogram Optimization

1. Problem Overview & Core Concept

The objective is to count all submatrices containing only 1s within a given binary matrix. A highly efficient approach conceptually transforms the matrix row-by-row into a histogram representation and utilizes optimizations to evaluate candidate rectangles efficiently.

Matrix-to-Histogram Transformation

Instead of scanning cells independently, we compress vertical contiguous blocks of 1s into heights. For each cell (i, j) , if $mat[i][j] == 1$, its cumulative height is $height[i][j] = height[i-1][j] + 1$; otherwise, it resets to 0.

Original Binary Matrix:

1	0	1	1
1	1	1	1
0	1	1	1

Histogram Representation:



1	0	1	1
2	1	2	2
0	2	3	3

2. Solution 1: Histogram with Linear Scan (Without Stack)

In this approach, we fix the bottom-right corner of the submatrices at row i and column r . We then shift a left pointer l from r backwards to 0, keeping track of the minimum histogram height encountered ($minu$). The number of valid rectangles ending at (i, r) with left-boundary l is bounded by this minimum height.

Time Complexity: $O(n \times m^2)$ where n is rows and m is columns.

Space Complexity: $O(n \times m)$ to store the histogram helper matrix.

C++ Implementation:

```
class Solution {
public:
    int numSubmat(vector<vector<int>>& mat) {
        int n=mat.size();
        int m=mat[0].size();
        int ans=0;
        vector<vector<int>> his(n,vector<int>(m));
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                if(mat[i][j]==0){
```

```

        his[i][j]=0;
    }

    else{
        if(i==0){
            his[i][j]=1;
        }

        else{
            his[i][j]=his[i-1][j]+1;
        }
    }
}
} //histogram bana liya

for(int i=0;i<n;i++){
    for(int r=0;r<m;r++){
        int minu=INT_MAX;
        for(int l=r;l>=0;l--){
            minu=min(minu,his[i][l]);
            ans+=minu;
        }
    }
}

return ans;
}
};

```

3. Solution 2: Optimal Monotonic Stack & Dynamic Programming

Instead of scanning backwards linearly and repeating the minimum height lookup, we optimize using a **Monotonic Stack**. The stack maintains indices of elements in increasing order of their histogram heights. This allows us to instantly find the previous smaller element index (*prev*).

DP Transition Logic:

- If no previous smaller element exists (stack is empty): All columns up to *r* can form rectangles bounded by the current height:

$$dp[r] = his[i][r] \times (r + 1)$$

- If a previous smaller element exists at index *prev*: The rectangles are split into two groups:
 1. Those bounded by the heights up to *prev*, which is precomputed in *dp[prev]*.
 2. Those from *prev + 1* to *r*, which are strictly bounded by the current height: $his[i][r] \times (r - prev)$.

$$\text{Formula: } dp[r] = dp[prev] + his[i][r] \times (r - prev)$$

Time Complexity: $O(n \times m)$ — Each column index is pushed and popped from the stack exactly once per row.

Space Complexity: $O(n \times m)$ for histogram and $O(m)$ for the DP array/stack.

C++ Implementation:

```
class Solution {
public:
    int numSubmat(vector<vector<int>>& mat) {
        int n=mat.size();
        int m=mat[0].size();
        int ans=0;
        vector<vector<int>> his(n,vector<int>(m));
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                if(mat[i][j]==0){
                    his[i][j]=0;
                }

                else{
                    if(i==0){
                        his[i][j]=1;
                    }

                    else{
                        his[i][j]=his[i-1][j]+1;
                    }
                }
            }
        } //histogram bana liya

        for(int i=0;i<n;i++){
            stack<int> st;
```

```

        vector<int> dp(m);
        for(int r=0;r<m;r++){

            while(!st.empty() && his[i][st.top()]>=his[i][r]){ //previous smaller store
krne ke liye
                st.pop();
            }

            if(st.empty()){
                dp[r]=his[i][r]*(r+1);
            }

            else{
                int prev=st.top();
                dp[r]=dp[prev]+his[i][r]*(r-prev);
            }

            st.push(r);
            ans+=dp[r];

        }
    }

    return ans;
}

};

```